

SkeletonSR (BasicSR): Pseudocode

Gist version ($\approx 60\%$ of the source)

Overview. A minimal PySR-style symbolic-regression engine. Expressions are binary/unary trees over variables and constants. The skeleton owns the tree representation, the evaluation budget, random tree generation, constant optimisation and the outer loop. The *policy* (in BLUE) decides how expressions are scored, chosen, mutated, crossed over, accepted and kept: `LOSS_FUNCTION`, `SELECTION`, `MUTATION`, `CROSSOVER`, `ACCEPTANCE`, `SURVIVAL`, `UPDATE_POPULATION`, `UPDATE_STATE!`.

Basics

- 1: **Node** ::= Const(value) | Var(feature) | Op(op, left, right | $\emptyset$)
- 2: **Individual** = (tree, loss, cost, complexity, lineage ids)
- 3: **HasBudget**: eval_count < max_evals **and** elapsed < timeout
- 4: **function** MAKEINDIVIDUAL(tree, parent)
- 5: **return** $\emptyset$ if no budget; eval_count += 1
- 6: (loss, cost) $\leftarrow$ `LOSS_FUNCTION`(tree); **return** Individual(tree, loss, cost, size(tree), parent)
- 7: **function** VALIDTREE(tree)
- 8: **return** size $\leq$ maxsize $\wedge$ depth $\leq$ maxdepth $\wedge$ per-operator size / nesting constraints hold

Initialisation

- 1: **function** INITIALIZEPOPULATION
- 2: **while** |pop| < population_size $\wedge$ HasBudget **do**
- 3: tree $\leftarrow$ small random tree (start at Const(0), attach three random operators to random leaves)
- 4: fall back to a depth- ≤ 3 random tree if invalid; score with MAKEINDIVIDUAL and add
- 5: **return** pop (or just the constant $\bar{y}$ if nothing could be scored)

Main loop

- 1: **function** FITSKELETONSR(X, y , config)
- 2: state $\leftarrow$ engine(X, y) + `INIT_STATE`(); populations $\leftarrow$ [INITIALIZEPOPULATION $\times n_{\text{pops}}$]
- 3: **for** iteration = 1 $\dots$ niterations **while** HasBudget **do**
- 4: **for each** population p **do**
- 5: **for** 1 $\dots$ ncycles_per_iteration **while** HasBudget **do**
- 6: `UPDATE_STATE!`(populations)
- 7: `EVOLVECYCLE!`(p)
- 8: `OPTIMIZEANDSIMPLIFY!`(populations[p])

```

9:         UPDATE_STATE!(populations)
10:        populations ← UPDATE_POPULATION(populations)    ▷ e.g. migration, hall of fame
11:    return archived expressions sorted by complexity
1: function EVOLVE_CYCLE!(p)
2:   pop ← populations[p]
3:   for 1... CYCLES_PER_POPULATION(pop) while HasBudget do
4:       if SHOULD_CROSSOVER(pop) then
5:           a, b ← SELECTION(pop), SELECTION(pop)    (distinct)
6:           children ← [MAKEINDIVIDUAL(t) for t in CROSSOVER(a, b)], keep those passing AC-
ACCEPTANCE
7:           pop ← SURVIVAL(pop, children) if any
8:       else
9:           parent ← SELECTION(pop)
10:          child ← MAKEINDIVIDUAL(MUTATION(parent))    ▷ skip cycle if mutation returns ∅
11:          if ACCEPTANCE(parent, child) then pop ← SURVIVAL(pop, [child])
12:   populations[p] ← pop

```

Per-population refinement

```

1: function OPTIMIZE_AND_SIMPLIFY!(pop)
2:   for each member in pop do
3:       optionally fold constant subtrees (Op(Const, Const) → Const)
4:       with prob. optimize_probability: refit the tree's constants with Nelder–Mead (a few
restarts),
       each objective call spending one eval of LOSS_FUNCTION; keep if loss improved

```